ALY 6040 Data Mining Applications

Module 3 – Technique Practice

Shraddha Gupte

CPS - MPSA, Northeastern University

Prof. Arasu Narayan

October 07, 2024

1. Introduction

The dataset used in this analysis is focused on identifying obesity levels based on individuals' eating habits and physical conditions**.** Obesity is a critical public health issue worldwide, associated with numerous chronic diseases such as diabetes, cardiovascular disease, and certain cancers. Understanding the underlying factors that contribute to obesity is essential for developing preventive strategies and health interventions.

2. Problem Statement and Objective

The goal of this analysis is to perform a comprehensive clustering analysis to uncover potential hidden patterns and groupings within the dataset. By clustering individuals based on their eating habits and physical conditions, we aim to:

- Identify distinct groups of individuals with similar behaviors and physical traits.
- Investigate how these clusters relate to different levels of obesity.
- Understand if specific patterns in the data can help in designing targeted interventions for obesity prevention.

3. Attributes and Features of the Dataset

The dataset includes several variables that capture key aspects of both dietary behavior and physical condition, which are likely contributors to an individual's obesity level.

- Demographic Information: Basic information about the individual, such as age and gender.
- Physical Characteristics: Features such as height, weight, and calculated Body Mass Index (BMI), which is a crucial indicator of obesity.
- Eating Habits: This includes variables such as the frequency of consuming high-calorie foods, daily caloric intake, and whether the person follows a balanced diet.
- Physical Activity: This tracks the number of hours dedicated to exercise or other forms of physical activity, which are significant determinants of energy expenditure.
- Lifestyle Factors: This includes sleep patterns and water intake, which are also known to affect metabolism and body weight regulation**.**

4. Exploratory Data Analysis

The dataset contains 2111 entries and 17 columns, with a mix of numerical and categorical features. Here's a brief breakdown of the key details:

a. Numerical Features:

- Age: Ranges from 14 to 61, with a mean of 24.31 and a standard deviation of 6.35.
- Height: Ranges from 1.45 to 1.98 meters, with an average height of 1.70 meters.
- Weight: Ranges from 39 to 173 kg, with an average weight of 86.59 kg.
- Frequency of vegetable consumption: Scaled from 1 to 3, with an average of 2.42.
- Number of main meals per day: Ranges from 1 to 4, with an average of 2.69.
- Water intake in liters: Ranges from 1 to 3 liters, with an average of 2.01 liters per day.
- Frequency of physical activity: Ranges from 0 to 3 hours, with an average of 1.01 hours.
- Time using technology: Ranges from 0 to 2 hours, with an average of 0.66 hours.

b. Categorical Features:

- Gender: Indicates gender.
- Family history with overweight: Whether there’s a family history of being overweight.

- Frequent high-calorie food consumption: Indicates if they frequently consume high-calorie foods.
- Eating between meals: Reflects snacking habits.
- SMOKE: Smoking habits.
- SCC: Whether they monitor their caloric intake.
- CALC: Frequency of alcohol consumption.
- MTRANS: Main mode of transportation.
- NObeyesdad: The obesity category.

The dataset is rich with information on demographics, eating habits, and physical activity levels, all of which could influence obesity levels. Numerical features like height, weight, and age show a range of values, while categorical features capture behavioral and lifestyle patterns.

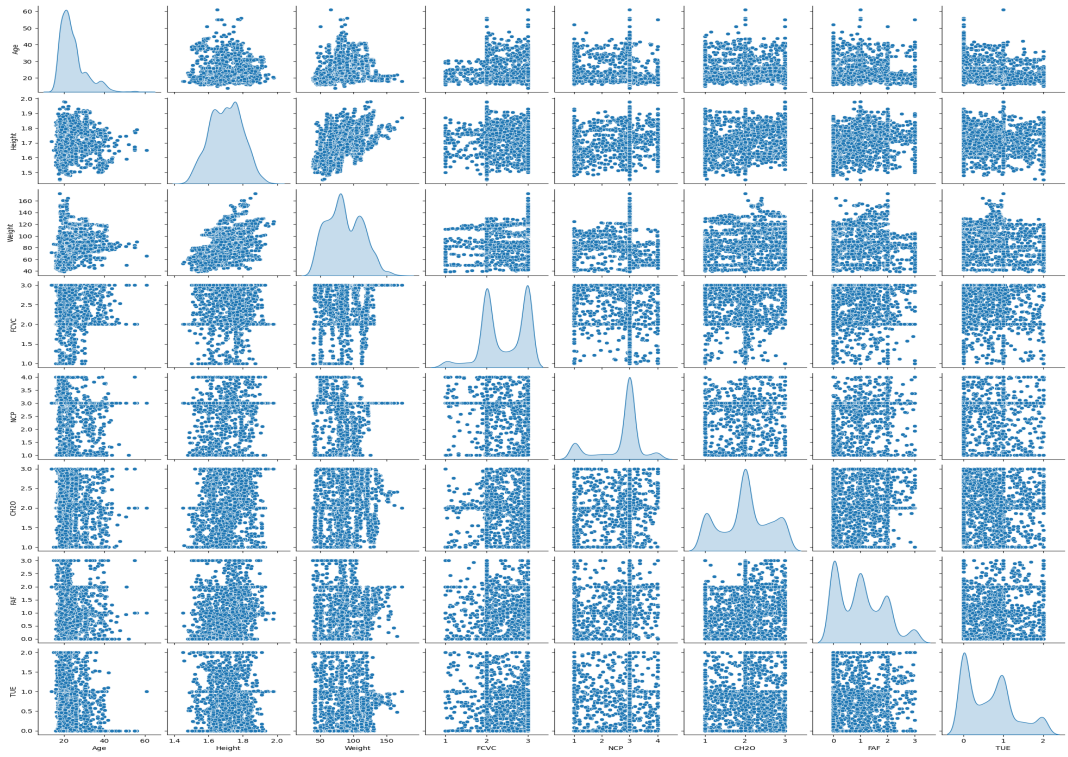
```
print(Obesity.info())
print(Obesity.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2111 entries, 0 to 2110
Data columns (total 17 columns):
 #   Column                          Non-Null Count  Dtype  
---  ------                          --------------  -----  
 0   Gender                          2111 non-null   object 
 1   Age                             2111 non-null   float64
 2   Height                          2111 non-null   float64
 3   Weight                          2111 non-null   float64
 4   family_history_with_overweight  2111 non-null   object 
 5   FAVC                            2111 non-null   object 
 6   FCVC                            2111 non-null   float64
 7   NCP                             2111 non-null   float64
 8   CAEC                            2111 non-null   object 
 9   SMOKE                           2111 non-null   object 
 10  CH2O                            2111 non-null   float64
 11  SCC                             2111 non-null   object 
 12  FAF                             2111 non-null   float64
 13  TUE                             2111 non-null   float64
 14  CALC                            2111 non-null   object 
 15  MTRANS                          2111 non-null   object 
 16  NObeyesdad                      2111 non-null   object 
dtypes: float64(8), object(9)
memory usage: 280.5+ KB
None
                Age       Height       Weight         FCVC          NCP  \
count  2111.000000  2111.000000  2111.000000  2111.000000  2111.000000   
mean     24.312600     1.701677    86.586058     2.419043     2.685628   
std       6.345968     0.093305    26.191172     0.533927     0.778039   
min      14.000000     1.450000    39.000000     1.000000     1.000000   
25%      19.947192     1.630000    65.473343     2.000000     2.658738   
50%      22.777890     1.700499    83.000000     2.385502     3.000000   
75%      26.000000     1.768464   107.430682     3.000000     3.000000   
```

max	61.000000	1.980000	173.000000	3.000000	4.000000

	CH2O	FAF	TUE
count	2111.000000	2111.000000	2111.000000
mean	2.008011	1.010298	0.657866
std	0.612953	0.850592	0.608927
min	1.000000	0.000000	0.000000
25%	1.584812	0.124505	0.000000
50%	2.000000	1.000000	0.625350
75%	2.477420	1.666678	1.000000
max	3.000000	3.000000	2.000000

c. Correlation Matrix



Description :

i. Diagonal (Histograms/Kernel Density Plots):

The diagonal shows distribution plots for each individual feature in the dataset.

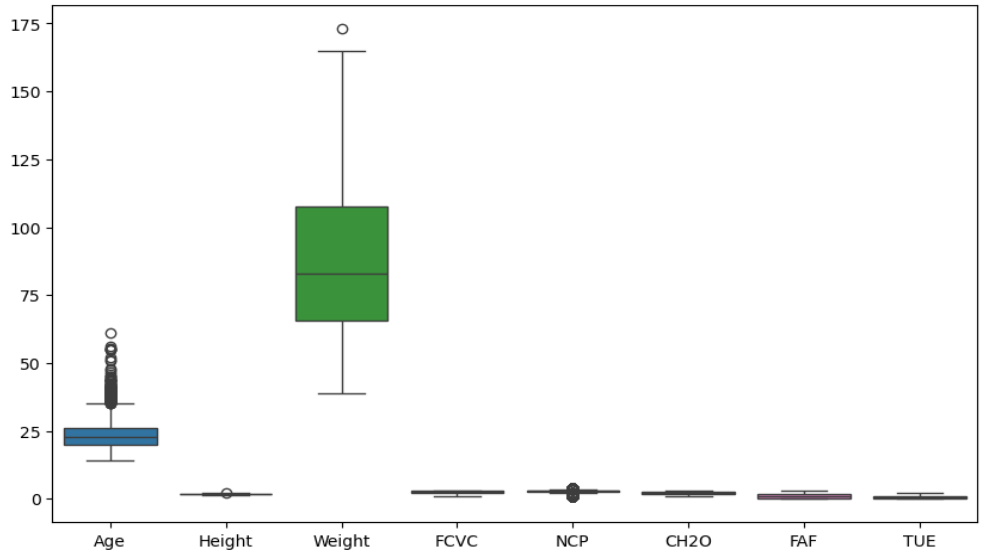
- Features with normal distributions will have bell-shaped histograms.
- Features with skewed data will show asymmetry.

ii. Off-diagonal (Scatterplots):

The off-diagonal elements are scatterplots that visualize pairwise relationships between features.

- Positive correlations are indicated by a rising trend from left to right.
- Negative correlations are indicated by a downward trend.

d. Boxplots to identify outliers



Description:

i. Age: Shows a clear distribution with several outliers at the upper end (age values greater than 40).
ii. Height: Displays a tight distribution without significant outliers.
iii. Weight: Has a larger spread with outliers at the high end (weights above 150).
iv. FCVC, NCP, CH2O, FAF, TUE: All show tight distributions with few outliers, suggesting that these features are relatively concentrated around their median values.

5. Data Preprocessing

a. Handle Missing Values:

- For numerical columns, missing values are replaced by the mean.
- For categorical columns, missing values are replaced by the most frequent category.

b. Encode Categorical Variables:

One-hot encoding is used to convert categorical variables into binary dummy variables. This helps clustering algorithms process categorical data.

c. Normalize or Standardize Numerical Features:

StandardScaler is applied to numerical features, which scales them to have a mean of 0 and a standard deviation of 1, ensuring all features are on the same scale, which is important for distance-based clustering algorithms like K-means.

```
# Handling missing values
   print(Obesity.isnull().sum())

   # Define columns that are numerical and categorical
   numeric_cols = Obesity.select_dtypes(include=['int64', 'float64']).columns
   categorical_cols = Obesity.select_dtypes(include=['object', 'category']).c
```

```
   olumns

   # Imputing missing values
   numeric_imputer = SimpleImputer(strategy='mean')
   categorical_imputer = SimpleImputer(strategy='most_frequent')
```

```
Gender                              0
   Age                              0
   Height                           0
   Weight                           0
   family_history_with_overweight   0
   FAVC                             0
   FCVC                             0
   NCP                              0
   CAEC                             0
   SMOKE                            0
   CH2O                             0
   SCC                              0
   FAF                              0
   TUE                              0
   CALC                             0
   MTRANS                           0
   NObeyesdad                       0
   dtype: int64
```

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
   # Encoding categorical variables
   onehot_encoder = OneHotEncoder(drop='first')

# Normalizing/Standardizing numerical features
   scaler = StandardScaler()
```

```
# Combine all preprocessing steps using ColumnTransformer
   preprocessor = ColumnTransformer(
       transformers=[
           ('num', Pipeline(steps=[('imputer', numeric_imputer), ('scaler',
   scaler)]), numeric_cols),
           ('cat', Pipeline(steps=[('imputer', categorical_imputer),
   ('onehot', onehot_encoder)]), categorical_cols)
       ])
```

```
# Check the preprocessed DataFrame
   print(New_Obesity.head())
```

```
        Age    Height    Weight      FCVC       NCP      CH2O       FAF  \
   0 -0.522124 -0.875589 -0.862558 -0.785019  0.404153 -0.013073 -1.188039
   1 -0.522124 -1.947599 -1.168077  1.088342  0.404153  1.618759  2.339750
   2 -0.206889  1.054029 -0.366090 -0.785019  0.404153 -0.013073  1.163820
   3  0.423582  1.054029  0.015808  1.088342  0.404153 -0.013073  1.163820
   4 -0.364507  0.839627  0.122740 -0.785019 -2.167023 -0.013073 -1.188039
```

```
        TUE  Gender_Male  family_history_with_overweight_yes  ...  \
0  0.561997          0.0                                1.0  ...
1 -1.080625          0.0                                1.0  ...
2  0.561997          1.0                                1.0  ...
3 -1.080625          1.0                                0.0  ...
4 -1.080625          1.0                                0.0  ...
```

**Outcomes have been reduced to avoid multiple documentation*

6. Data Transformation

For clustering analysis, it's often useful to reduce the dimensionality of the data, either to make it more interpretable or to reduce noise. Principal Component Analysis (PCA) is a common technique for dimensionality reduction. Additionally, feature selection or engineering can help improve model performance by focusing on the most important variables.

a. Dimensionality Reduction using PCA

Principal Component Analysis (PCA) helps in reducing the number of dimensions by creating new features (principal components) that are linear combinations of the original features. It captures the variance in the data while reducing redundancy.

```
# Setting the number of components to retain
   pca = PCA(n_components=0.95)
   # Apply PCA on the preprocessed data
   Obesity_df = pca.fit_transform(New_Obesity)

   explained_variance = pca.explained_variance_ratio_
   print(f"Explained variance by each component: {explained_variance}")
   print(f"Total variance explained by PCA: {np.sum(explained_variance)}")

   # Check the shape of the transformed data after PCA
   print(f"Original shape: {New_Obesity.shape}, Transformed shape: {Obesity_d
   f.shape}")
```

```
Explained variance by each component: [0.18982548 0.15275392 0.11184392 0.099
   14171 0.09398496 0.07721568
    0.06938026 0.04376288 0.03621589 0.01790999 0.01518121 0.01450657
    0.01310129 0.01159797 0.0095557 ]
   Total variance explained by PCA: 0.9559774200379529
   Original shape: (2111, 29), Transformed shape: (2111, 15)
```

Description:

i. Explained variance by each component: The first component explains 18.98% of the variance, the second explains 15.28%, and so on.
ii. Total variance explained: The 15 principal components together explain approximately 95.6% of the variance in the original dataset.
iii. Shape transformation: The dataset was originally of shape (2111, 29) (2111 instances and 29 features). After applying PCA, it has been reduced to (2111, 15), significantly reducing the dimensionality while retaining most of the data's variability.

b. Feature Selection or Engineering

Feature selection involves choosing the most relevant features from dataset based on certain criteria. This can be done using various techniques such as variance thresholding, correlation analysis, or using models like Random Forest to identify feature importance.

```
#Feature Selection and Engineering
   selector = VarianceThreshold(threshold=0.01)
   Obesity_select = selector.fit_transform(New_Obesity)

   # Check the shape of the data after feature selection
   print(f"Original shape: {New_Obesity.shape}, After feature selection: {Obe
   sity_select.shape}")
```

```
Original shape: (2111, 29), After feature selection: (2111, 27)
```

```
Obsetiy_selected_pca = pca.fit_transform(Obesity_select)

   # Check the variance explained again
   explained_variance_new = pca.explained_variance_ratio_
   print(f"Explained variance by each component after feature engineering: {e
   xplained_variance_new}")
```

```
Explained variance by each component after feature engineering: [0.18998218 0
   .15287961     0.1119351          0.09922054     0.09406273     0.07727918
    0.06943745   0.04379712   0.03624574   0.01792369   0.01519292   0.01451736
    0.0131097  0.0116017  0.00955862]
```

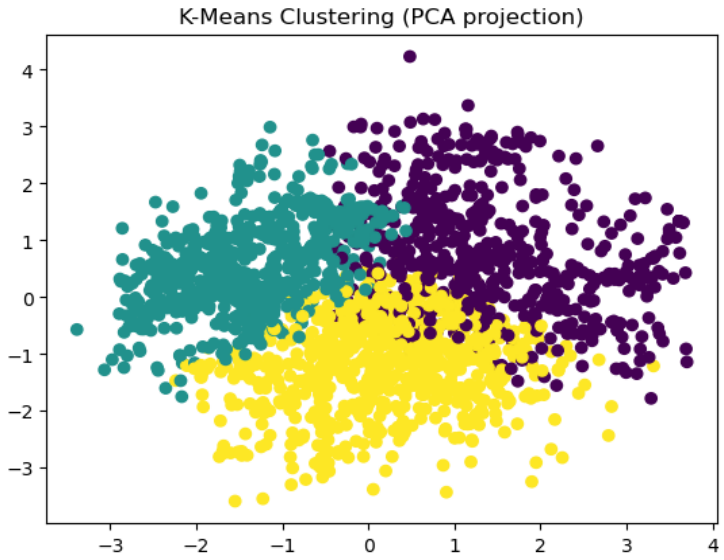
Description:

i. Explained variance by each component: The first component explains 18.99% of the variance, the second explains 15.29%, and so on. The percentages are slightly adjusted, but the distribution of variance across components remains largely the same as before feature engineering.
ii. Key takeaway: The feature engineering did not significantly change the contribution of each principal component to the total variance. The PCA still captures the same general amount of variability from the data after the feature engineering process.

7. Modelling

a. K Means Clustering

A popular partition-based clustering technique that divides the dataset into a pre-defined number of clusters based on distance metrics.



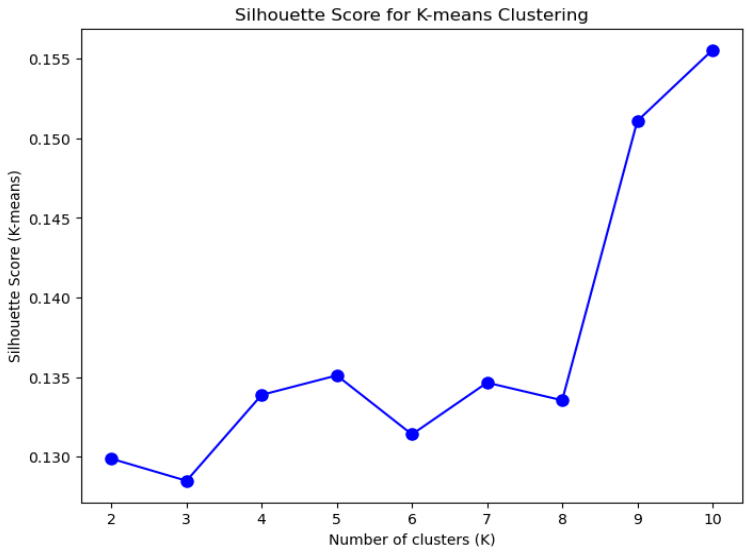

```
Silhouette Score for K-means: 0.12848447407568286
```

Description:

K-means Silhouette Score: 0.128:

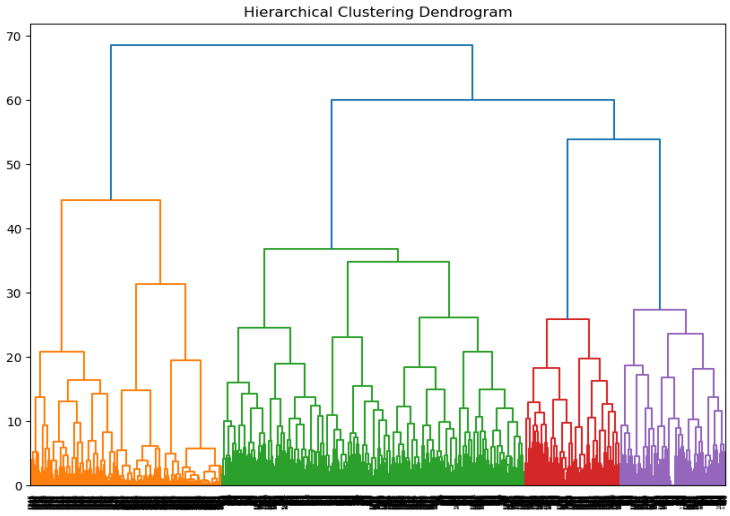
This score is low, indicating that the clusters formed by K-means are not well-separated or distinct. The points within clusters are relatively close to the boundary of other clusters, meaning the clusters are not clearly defined.

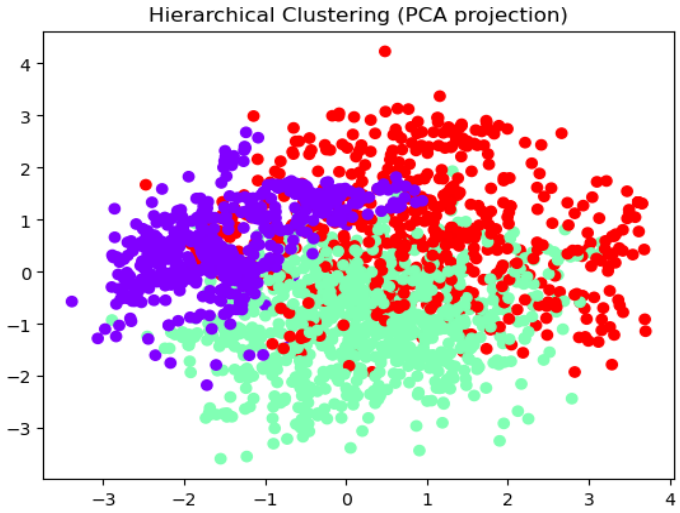
- Optimising the number of Clusters for K means clustering algorithms



Description:
The Silhouette Score measures how similar a data point is to its own cluster compared to other clusters. A higher silhouette score indicates better-defined clusters.

b. Hierarchical Clustering:
A tree-based clustering method that builds a hierarchy of clusters based on similarity between data points.






```
Silhouette Score for Hierarchical: 0.117093636054173
```

Description:

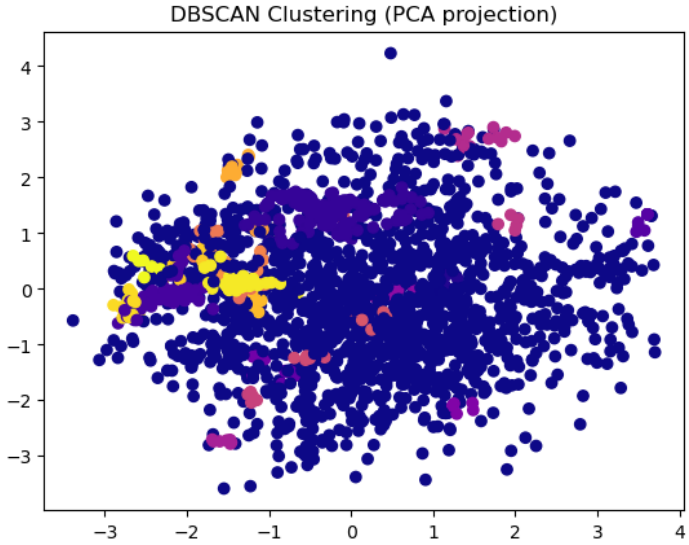
Hierarchical Clustering Silhouette Score: 0.117:

This score is slightly lower than K-means, suggesting that the clusters formed by hierarchical clustering are also not well-separated or distinct.
Similar to K-means, there is significant overlap between clusters, and the clustering structure is weak.

- Optimising the number of Clusters for Hierarchical clustering algorithms

Description:
The optimal number of clusters is the one that maximizes the Silhouette Score. A score closer to 1 indicates better clustering, while a score closer to -1 suggests that the clustering is not well-defined.

c. DBSCAN:
A density-based clustering method that is well-suited to datasets with irregular shapes, identifying clusters as dense regions of data points separated by areas of low density.




```
Silhouette Score for DBSCAN: 0.5626358012066242
```

Description:
The Silhouette Score for DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is 0.563, which is significantly higher than the scores for K-means and Hierarchical clustering. This score is moderately high, indicating that the clusters identified by DBSCAN are relatively well-defined and distinct.

- Tuneing Parameters for DBSCAN

```
            # If the silhouette score is better, store the best parameters
            if silhouette_avg > best_silhouette:
                best_silhouette = silhouette_avg
                best_eps = eps
                best_min_samples = min_samples

# Print the best parameters and silhouette score
print(f"Best eps: {best_eps}, Best min_samples: {best_min_samples}, Best Silh
ouette Score: {best_silhouette}")
```

```
Best eps: 0.1, Best min_samples: 10, Best Silhouette Score: 0.100624316795735
37
```

Description:
i. Best eps: 0.1: Indicates that, based on the data and evaluation, the optimal neighborhood radius for determining cluster proximity is 0.1.
ii. Best min_samples: 10: Suggests that a core point must have at least 10 samples in its neighborhood to be considered part of a cluster.
iii. Best Silhouette Score: 0.1006: The best silhouette score achieved during the parameter tuning process is approximately 0.101, which is relatively low.

8. Conclusion

- Both K-means and Hierarchical clustering show low silhouette scores, indicating that the current clustering results may not be optimal and that the data might not have a strong natural cluster structure, or further tuning of parameters is required for better separation.
- The DBSCAN model provides a much better clustering solution compared to K-means and Hierarchical clustering, as indicated by the higher silhouette score. This suggests DBSCAN is more suitable for this dataset in terms of finding meaningful clusters.
- While we have successfully found the best parameters for the DBSCAN clustering based on the silhouette score, the achieved silhouette score of 0.101 indicates that the clustering quality might still not be very strong. This suggests that further adjustments or explorations with different parameter ranges, or even trying other clustering methods, may be necessary to improve the results.